# Real-Time Leaderboard — Architecture & Leadership Note

Answers to the design, performance, and leadership questions from the assignment brief.

**Contents**



## 1. Architecture

### Why did you split it into modules this way?

The brief asks for two independent pieces — a score generator and a leaderboard — so I wanted "independent" to actually mean something, not just be true by convention. The way I did that:

There's a tiny `core` module with nothing in it but three data classes ( `ScoreEvent` , `Player` , `LeaderboardEntry` ). Both `engine` (the fake match generator) and `leaderboard` (the ranking logic) depend on `core` , but not on each other. Neither one can import the other even if someone tried — there's no line to add to a build file that would make that compile. That's a stronger guarantee than "please don't couple these" written in a comment.

The leaderboard module's public entry point takes any `Flow<ScoreEvent>` — it doesn't know or care that the events came from a fake random generator. On the app side, the ViewModel is handed a `ScoreEventSource` interface, not the concrete engine class. So if this were ever

hooked up to a real backend instead of the fake generator, that's one new class implementing one interface, wired in from one place — nothing in the ranking logic or the UI would need to change.

In the terms the brief uses for Part 2: `engine` is the data/engine layer, `leaderboard` is the domain logic, and the app module (ViewModel + Compose UI) is the UI layer.

### Where does the ranking logic live, and why?

Entirely inside the `leaderboard` module, in a plain function called `RankingRules.rank(...)` — no coroutines, no Android, just a list in and a ranked list out. The ViewModel never sorts or ranks anything; it just forwards whatever the leaderboard module produces. I did this mainly so the ranking rules could be tested with plain JUnit and nothing else, and so there's exactly one place to look if a ranking bug ever shows up — not "somewhere between the ViewModel and the UI."

### Why four Gradle modules instead of two?

Fair question, since the brief describes two modules and gives a pretty tight time budget. The extra `core` module isn't really "extra" — it's the only way to let `engine` and `leaderboard` share the event data type without one of them depending on the other, which is the whole point of keeping them independent. I didn't reach for anything beyond that: no DI framework, no repository-pattern layers, nothing speculative. It's four modules, but two of them ( `engine` , `leaderboard` ) are the real deliverables, one ( `core` ) is three data classes, and one ( `app` ) is the Android app every project needs anyway.

### Compose or XML, and why?

Compose. The leaderboard is a list where the contents, order, and highlighting all change constantly — that's exactly the case Compose's `LazyColumn` with keyed items and built-in item-placement animations is designed for. Doing the same thing in XML would mean writing a `DiffUtil.Callback` and a custom `ItemAnimator` by hand to get the same "only the changed rows animate" behaviour that Compose gives for free.

## 2. Performance & lifecycle

### How do you avoid blocking the UI thread?

All the actual work — the engine's random delay/score-picking loop, and the leaderboard's sort/rank step — runs on `Dispatchers.Default`. The UI side only ever reads an already-computed list off a `StateFlow`; it never sorts or waits on anything itself.

### How do you avoid unnecessary recompositions?

Two separate things, because they're actually two separate problems:

- **Not re-computing when nothing changed.** The leaderboard flow has `distinctUntilChanged()` on it, so a duplicate or no-op event that doesn't actually move anyone's rank never gets sent onward. This is also what stops the flickering the brief specifically warns about.
- **Not re-drawing rows that didn't change.** Even with a genuinely new list, Compose shouldn't re-render every row just because one player's score moved. So the row model is a plain data class of primitives (easy for Compose to treat as stable), the list is exposed as an `ImmutableList` instead of a regular `List` (which Compose can't otherwise prove is stable), and each row in the `LazyColumn` is keyed by the player's ID so reordering animates a row into its new spot instead of tearing it down and rebuilding it.

### How do you avoid memory leaks?

The three non-UI modules (`core`, `engine`, `leaderboard`) don't import a single Android class — there's nothing there that could hold a reference to an Activity or a View. On the app side, the only long-running coroutine lives in the ViewModel's own scope, which gets cancelled automatically when the ViewModel is cleared.

### What happens on screen rotation?

Nothing visible — the state lives in the ViewModel, which survives the rotation. The screen gets torn down and rebuilt, but it just reconnects to the same ongoing data stream and shows the current value immediately.

### What happens when the app goes to the background?

Collection stops once the screen isn't visible, and after five seconds with nobody watching, the underlying flow gets cancelled too — so the fake engine actually stops generating events while nobody can see them, rather than quietly burning battery in the background. The five-second delay is just there so a quick rotation or app-switcher glance doesn't cause a visible restart.

Coming back to the foreground reconnects and shows the same standings the leaderboard was at before it was backgrounded, then keeps updating live from there — it does not reset anyone's score or replay the match from the start. That specifically requires the running score state to live on the engine instances themselves (`RandomScoreEngine`, `DefaultLeaderboardEngine`), not inside the flow that gets cancelled — otherwise every reconnect would silently restart the whole simulated match from zero, which is a real bug I hit and fixed while testing this on-device (both engines now guard that state with a `Mutex` in case of concurrent collectors, and there are regression tests in both modules covering the exact "detach, then re-attach" scenario).

### How would this scale to 1,000 users? To 100,000?

At 1,000 users, what's here already works fine — resorting a map of 1,000 entries on every update is well under a millisecond, so there's no reason to complicate it.

At 100,000, I'd change three things, roughly in this order:

1. Only ever compute/show the top N (say, top 100) plus "your own rank" as a separate cheap lookup — nobody's scrolling through 100,000 rows anyway.
2. Stop resorting the whole list on every update and use a data structure that supports moving one entry in roughly log(n) time instead.
3. Move the aggregation itself onto a server and have the app just subscribe to a small stream of "the top of the board changed" events. At that scale, keeping every user's live score in memory on a phone stops making sense regardless of the algorithm — and because the app only depends on the `ScoreEventSource` interface, this is a new implementation of that interface, not a rewrite of the leaderboard or the UI.

## 3. Leadership & ownership

### What trade-offs did you consciously make?

- **Four Gradle modules instead of two** — more build-file ceremony, in exchange for the independence between engine and leaderboard being enforced by the compiler rather than by discipline.
- **No dependency-injection framework** — a single hand-written object wires everything together. That's the right amount of structure for one screen; it wouldn't be if this app grew past a handful of screens.
- **A full in-memory resort per update** rather than an incremental data structure — simpler and provably correct at the scale this actually runs at (documented above as the first thing to change before 100k users).
- **The session seed is based on when the app launches**, not a fixed constant — every run is a fresh, different match, which felt like the more honest simulation of a live game, at the cost of not being trivially replayable from the UI alone (it is replayable from a debugger, since the seed is just a value logged at startup).

## 4. Code review — if a mid-level dev had written this

The brief asks for this as if reviewing someone else's first pass at the same system. Here's what I'd actually flag, and why each one matters:

### MUST FIX Ranking recomputed on every tick, no de-duplication before it reaches the UI

Even a duplicate or no-op event pushes a brand-new list to the screen — this is the literal cause of the flickering the brief warns about. Fix: derive the ranking functionally and drop `distinctUntilChanged()` in before it leaves the domain layer.

### MUST FIX The engine is driven by a `Handler.postDelayed` in the ViewModel instead of a coroutine `Flow`

This is exactly the "timer in the UI pretending to be real-time" pattern the brief calls out by name. It also means the engine isn't actually reusable or testable on its own — you can't unit test it without dragging the UI layer along.

### MUST FIX Tied scores get sequential ranks (1, 2, 3) instead of a shared rank (1, 2, 2)

This is a stated business rule, not a style choice — it'll be graded as flat-out wrong, not just untidy.

### IMPROVEMENT List rows are keyed by index instead of by user ID

Once the list reorders — which happens almost every tick — index-based keys make Compose think a reordered row is "the same slot, new content," so the rank-movement animation either animates the wrong row or doesn't fire at all.

### IMPROVEMENT A new unseeded `Random()` gets created on every tick

Reseeding constantly from the clock breaks the "same seed replays the same match" requirement, and is measurably worse randomness besides — one seeded generator held for the whole session is both simpler and correct.

### IMPROVEMENT Sorting happens inline on whatever dispatcher the collector is on

Fine with ten players, but it silently depends on the caller never collecting from the main thread. That's the kind of assumption that turns into a jank bug the moment someone adds a second collector.

### TECH DEBT The leaderboard module imports the concrete engine class directly, not the interface

Works today, but it's the exact coupling between the "two independent modules" the brief is checking for. The day a real backend replaces the fake engine, this forces a change inside the domain module instead of just at the composition root.

### TECH DEBT No tests for ranking edge cases (all-zero scores, one player, an empty list)

The ranking function is about the cheapest possible thing to test exhaustively. Leaving edge cases untested here isn't saved effort, it's just unclaimed risk.

## 5. If this had to ship in 7 days

### What's non-negotiable, and what would you cut?

**Keeping:**

- Correct ranking, with tests — this is the one part that's graded as outright right or wrong.
- A real Flow-based pipeline end to end, no timers pretending to be real-time.
- Nothing blocking the main thread.
- The interface seam between engine and leaderboard — cheap now, expensive to retrofit later.

**Cutting or deferring:**

- Animation polish beyond the two effects already in place.
- CI/lint tooling — valuable, but doesn't block shipping something correct.
- Anti-cheat — genuinely a "once there's a real backend to be authoritative against" problem.
- Any persistence — there's no offline requirement for this feature.

### How would you split the work?

| Who | What they'd own |
|---|---|
| Junior dev | The Compose row layout and theming, wiring up the fake player list, and the straightforward ranking edge-case tests — under review. |
| Mid-level dev | Building the engine and leaderboard modules against an interface I hand them up front, plus their tests. |
| Me (lead) | Defining the module boundaries and interfaces before anyone starts, reviewing the ranking algorithm for correctness, the recomposition-avoidance approach, final integration, and the review above. |

## 6. What I'd do with more time

- Add a fake `ScoreEventSource` / `LeaderboardEngine` and write proper ViewModel tests — right now the ViewModel itself isn't directly unit-tested, only the modules underneath it.
- Replace the full resort with an incrementally-updated structure, so the code itself demonstrates the 100k-user story instead of just this document.
- Actually wire up ktlint/detekt in CI instead of leaving it as a proposal.
- Give `ScoreEventSource` an error channel — right now it's a happy-path-only stream, which is fine for a fake engine but wouldn't be for a real network source.

## 7. A few extra ideas (optional section)

The assignment allows picking one optional item to go deeper on — I went with real unit tests for the ranking logic (that's in the repo, not here). The three below are just proposals, not code.

### CI / quality checks

I'd add ktlint and detekt as a GitHub Actions job on every pull request, alongside the existing test suites:

```
name: CI
on: [pull_request]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '17' }
      - run: ./gradlew ktlintCheck detekt test
```

### Anti-cheat ideas for live tournaments

- Make the score the server's responsibility, not the client's — the client should only ever relay server-signed events, never compute its own deltas.
- Flag score jumps or update rates that are statistically way outside what's normal for a given game mode.
- Since the fake engine is already seed-driven and deterministic, the same idea applied server-side (logging the seed and inputs, not just the outputs) would let a disputed match be replayed exactly for review.

### Production-readiness

- Pagination for large boards — top N plus "your rank," as described in the scaling answer above.
- Swap the fake engine for a WebSocket-backed implementation of `ScoreEventSource` — no other code needs to change.
- Crash and analytics hooks for visibility once this is actually running in production.

- Feature-flag the animations so they can be switched off instantly if they misbehave on low-end devices.